



Unlocking Object Functionality II in Python

Section 1: Learn

What More Can Objects Do?

In the previous lesson, we covered **creating objects, using methods, and inheritance**.

Now, let's explore **advanced object functionality** like:

- **Encapsulation** (hiding data for security)
- **Polymorphism** (using the same method in different ways)
- **Dunder (Magic) Methods** (special methods that enhance functionality)

Why Learn Advanced Object Functionality?

- **Improves Code Security:** Encapsulation prevents direct modification of important data.
- **Enhances Reusability:** Polymorphism allows flexible method usage.
- **Makes Objects More Powerful:** Dunder methods customize object behavior.

Fun Fact: OOP and Real Life 🚀

Did you know **video game characters** use OOP concepts?

For example, a **“Player” class** can have multiple types (Mage, Warrior), each with unique abilities—just like polymorphism!

Section 2: Practice

1. Encapsulation: Protecting Data with Private Variables

Encapsulation **restricts access** to certain object properties.



```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance # Private variable

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance

account = BankAccount(1000)
account.deposit(500)
print(account.get_balance()) # Output: 1500
```

◆ Key Takeaways:

- `__balance` is **private** and cannot be accessed directly (`account.__balance` will give an error).
- Use **getter methods** (`get_balance()`) to access private data.

2. Polymorphism: Same Method, Different Behavior

Polymorphism allows **methods to be redefined in child classes**.

```
class Bird:
    def sound(self):
        return "Some generic sound"

class Sparrow(Bird):
```



```
def sound(self):  
    return "Chirp Chirp!"  
  
class Crow(Bird):  
    def sound(self):  
        return "Caw Caw!"  
  
birds = [Sparrow(), Crow()]  
  
for bird in birds:  
    print(bird.sound())
```

Output:

```
Chirp Chirp!  
Caw Caw!
```

◆ Key Takeaways:

- `sound()` method works **differently** in different child classes.
- This makes code **flexible and reusable**.

3. Magic Methods (Dunder Methods)

Dunder (Double Under) methods **customize object behavior**.

Example: `__init__()`, `__str__()`, `__len__()`

```
class Book:  
    def __init__(self, title, pages):  
        self.title = title
```



```
self.pages = pages

def __str__(self):
    return f"Book: {self.title}, Pages: {self.pages}"

def __len__(self):
    return self.pages

book = Book("Python Mastery", 450)
print(book) # Calls __str__()
print(len(book)) # Calls __len__()
```

Output:

```
Book: Python Mastery, Pages: 450
450
```

◆ Key Takeaways:

- `__str__()` makes `print(book)` return a readable string.
- `__len__()` makes `len(book)` return the number of pages.
- These **make objects behave like built-in data types!**

Try This Yourself:

1. **Create a Car class** where `__len__()` returns the number of wheels.
 2. **Use encapsulation** to hide and protect a student's exam marks.
-



Section 3: Know More

Frequently Asked Questions

Q1: What's the difference between Encapsulation and Data Hiding?

Encapsulation **bundles data and methods**, while data hiding **restricts direct access**.

Q2: Can I access private variables outside the class?

Yes, using **name mangling**:

```
print(account._BankAccount__balance) # Not recommended!
```

Q3: Are dunder methods mandatory?

No, but they **make objects more powerful**.

Q4: Can polymorphism work without inheritance?

Yes! It works with **any two unrelated classes** as long as they have the same method name.

Q5: What other dunder methods exist?

Some useful ones:

- `__add__()` – Customizes `+` operator.
- `__eq__()` – Customizes `==` operator.
- `__getitem__()` – Allows indexing like a list.

Mastering these advanced object functionalities will **help you write efficient, reusable, and flexible code!** 🚀